Evaluating Machine Learning Algorithms for Enhancing Network Intrusion Detection Systems: A Comparative Study

Gunay Abdiyeva-Aliyeva
Department of Information Technology
Baku Higher Oil School
Baku, Azerbaijan
gunay.abdiyeva@bhos.edu.az

Naila Allakhverdiyeva
Department of Information Technology
Baku Higher Oil School
Baku, Azerbaijan
naila.allakhverdiyeva@bhos.edu.az

Nihad Alili
Department of Information Technology
Baku Higher Oil School
Baku, Azerbaijan
nihad.elili@bhos.edu.az

Ilkin Balazade
Department of Information Technology
Baku Higher Oil School
Baku, Azerbaijan
ilkin.balazade@bhos.edu.az

2025 13th International Symposium on Digital Forensics and Security (ISDFS) | 979-8-3315-0993-4/25/$31.00 ©2025 IEEE | DOI: 10.1109/ISDFS65363.2025.11012084

***Abstract*— Network Intrusion Detection Systems (NIDS) play crucial role for maintaining the security of modern network infrastructures. With the increasing sophistication of cyber-attacks, traditional signature-based methods have become insufficient. Machine Learning (ML) techniques offer a promising alternative, providing the ability to detect novel threats by learning from network traffic patterns. This article provides a comparative analysis of different ML algorithms applied to NIDS, evaluating their performance based on key metrics such as accuracy, precision, recall, and F1-score.**

***Keywords*—Random Forest, Decision Tree, XGBoost, NIDS, NSL, KDD**

I. Introduction

As network infrastructures grow and become more complex, the necessity for efficient security measures increases even more. Network Intrusion Detection Systems (NIDS) are crucial components that monitor network traffic for suspicious activities and potential threats [1]. Traditional NIDS depend mostly on signature-based detection methods, which identify known, as well as unknown, threats [2]. This article examines the performance of different ML algorithms on the NSL-KDD dataset, which is a standard network intrusion detection benchmark dataset.

II. ML Techniques for NIDS

Several ML algorithms have been used in NIDS, each with its strengths and weaknesses. This brief overview is given of the most commonly used techniques.

Decision Trees are basic but effective algorithms that divide the data based on according to feature values to label the output. They are easy to understand and can accept, both numerical and categorical data. However, they tend to overfit, especially with noisy data.

Random Forest is the ensemble learning which is constructed by the aggregation of numerous decision trees to obtain the best results. It reduces the problem of overfitting by computing the average of the predictions of the independent trees as well as it is less affected by the noise in data. However, it can be computationally expensive and less interpretable than a single decision tree [3].

K Nearest Neighbors (KNN) is a very simple instance-based learning algorithm that classifies the data points based on the proximity to other data points in the feature space. It is quite easy to understand and implement but is computationally expensive and highly sensitive to k and the distance metric.

Naive Bayes is a probabilistic classifier that is built on the concept of Bayes' theorem with the assumption of automatic features. It is easy to implement, and performs fairly well for high-dimensional data. Nonetheless, its strong assumption of independence assumption in general does not hold in real life.

Neural Networks (NN) are flexible models with the ability to identify complex patterns from data. Although they can be highly precise, but require large amounts of data and computational resources. Furthermore, they are often viewed as “black boxes” due to their lack of interpretability.

Gradient Boosting (XGBoost) is an ensemble method of buildings models sequentially, in which subsequent model corrects the previous model errors. Although it can provide high performance, it prones to overfitting and requires careful hyperparameters tuning [4].

LinearSVC is a linear classifier that identifies the best hyperplane to distinguish classes in feature space. It performs well with high-dimensional data efficiently and on linearly separable data. However, it can be challenged by nonlinear relationships and requires precise tuning of the regularization parameters to avoid overfitting and obtain the best results.

III. Experimental Setup

A. Data Preprocessing

The data preprocessing is done on the numerical features and encoding of categorical features is done. In addition, tests were conducted on training and test sets to determine the ability of the models to detect attacks.

B. Evaluation Metrics

The assessment of the ML models in the process of the network attack recognition is very important in regard to the allowance of the model having precisions in distinguishing between the normal and the actual threats. The methods below measure the model’s effectiveness:

979-8-3315-0993-4/25/$31.00 ©2025 IEEE

Authorized licensed use limited to: Access provided by VTU consortium. Downloaded on October 02,2026 at 07:51:54 UTC from IEEE Xplore. Restrictions apply.

Accuracy. The rate at which the correct instances are classified against the total instances is referred to as the Accuracy. It gives a general impression of the performance of the model over all the classes.

$$Accurancy = \frac{TP+TN}{TP+TN+FP+FN} \quad (1)$$

Where:

- TP (True Positive) signifies the malicious instances which are detected correctly.
- TN (True Negative) indicates the normal instances which are identified correctly.
- FP (False Positive) means the normal instances that are mistakenly detected as malignant.

FN (False Negative) portrays the malignant instances which are mistaken as normal by error.

Precision. Precision measures the number of true positive predictions to all positive predictions against all positive predictions given by the model. High precision reflects that the model has a low rate of false alarms, a key consideration for attack detection so as not to avoid unnecessary alerts.

$$Precision = \frac{TP}{TP+FP} \quad (2)$$

Recall. Recall, also known as sensitivity or true positive rate, measures the ratio of true positive predictions to all actual positive instances.

$$Recall = \frac{TP}{TP+FN} \quad (3)$$

F1 score. The F1 score is a conversational average of precision and recall and thus provides a single metric balancing both. It is especially effective with imbalanced class distribution as it evaluates both false positives and false negatives.

$$F1 - Score = 2 * \frac{Precision*Recall}{Precision+Recall} \quad (4)$$

Specificity: Specificity, which is also known as the true negative rate, expresses the percentage of true negative predictions among the total of actual negative instances. By indicating high specificity, the model asserts that it can distinguish between the good and the malicious instances efficiently.

$$Specificity = \frac{TN}{TN+FP} \quad (5)$$

Area Under the ROC Curve (AUC-ROC). The true positive rate (recall) and false positive rate (1 - specificity) at different threshold settings being conversely exchanged are outlined by the AUC-ROC examined. The higher the AUC-ROC value, the better the model is at differentiating the classes.

Confusion Matrix: The confusion matrix is the actual result and the predicted result classification tabular that the model makes.

TABLE 1. CONFUSION MATRIX IS A TABULAR REPRESENTATION INDEED

	Predicted Positive	**Predicted Negative**
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

Balanced Accuracy. Balanced accuracy is a measure that considers class imbalance by averaging the recall for each class. It is an important performance metric for datasets that have a significantly larger number of normal instances compared to malicious ones.

$$Balanced\ Accuracy = \frac{Recall\ (Positive\ Class)+Recall\ (Negative\ Class)}{2} \quad (5)$$

C. Evaluation Procedure

This process outlines the steps for evaluating ML models in detecting network intrusions by splitting the dataset, training models, making predictions, calculating performance metrics, and analyzing results to identify the most effective algorithms [6].

- Data Sharding: Split the dataset into training and testing sets so that the models are evaluated on the data not used in the training phase hence simulating real-world performance.
- Model Learning: Every ML model is taught with the training set using the corresponding hyperparameters and structure.
- Model Inference: The learned model will assign the necessary class labels in the test set.
- Metric Evaluation: The labels which have been predicted earlier are matched with those of the test set thus, the previously mentioned evaluation metrics are calculated.Result Analysis: The metrics are analyzed to determine the strengths and weaknesses of each model and highlight which algorithms are most effective in detecting network intrusions.

D. Importance of Evaluation Metrics in NIDS

When detecting cyberattacks, the choice of evaluation metrics is critical for the following reasons:

- High risk: Misclassifying malicious instances as normal instances (false positives) may lead to security vulnerabilities, while misclassifying normal instances as malicious instances (false negatives) may cause unnecessary alerts and consume valuable resources.
- Imbalanced data: Cyberattack datasets often reflect an unbalance between the regular and the malicious instances, which is why metrics like precision, recall, and F1 score become more informative than precision alone.
- Real-time Detection: NIDS require real-time or near-real-time performance, so the evaluation should also consider

Authorized licensed use limited to: Access provided by VTU consortium. Downloaded on October 02,2026 at 07:51:54 UTC from IEEE Xplore. Restrictions apply.

the computational efficiency of the models alongside their predictive performance.

By comprehensively evaluating ML models using these metrics, we can identify the most effective techniques for network intrusion detection, ensuring robust and reliable security for network infrastructures [7].

IV. Implementation in Python

A. Loading and Preprocessing the Dataset

Here's how it can be done in Python:

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder, OneHotEncoder

url = "https://raw.githubusercontent.com/defcom17/NSL_KDD/master/KDDTrain+.txt"
column_names = ["protocol_type", "service", "flag", "label"]
df = pd.read_csv(url, header=None, names=column_names)

# Handle categorical features using OneHotEncoder
categorical_columns = ['protocol_type', 'service', 'flag']
ohe = OneHotEncoder(handle_unknown='ignore')
ohe_df = pd.DataFrame(ohe.fit_transform(df[categorical_columns]).toarray(), columns=ohe.get_feature_names_out(categorical_columns))

# Reset index of the original DataFrame to ensure compatibility
df = df.drop(categorical_columns, axis=1).reset_index(drop=True)
df = pd.concat([df, ohe_df], axis=1)

label_encoder = LabelEncoder()
df['label'] = label_encoder.fit_transform(df['label'])
X = df.drop('label', axis=1)
y = df['label']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Convert column names to strings before scaling
X_train.columns = X_train.columns.astype(str)
X_test.columns = X_test.columns.astype(str)

# Now scale the numerical features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Fig. 1. Loading and Preprocessing the Dataset

B. Training and Evaluating Different ML Models

Here's how it can be done in Python:

```python
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.svm import SVC, LinearSVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.metrics import classification_report

models = {
    'Decision Tree': DecisionTreeClassifier(),
    'Random Forest': RandomForestClassifier(),
    'KNN': KNeighborsClassifier(),
    'Naive Bayes': GaussianNB(),
    'Neural Network': MLPClassifier(max_iter=300),
    'Gradient Boosting': GradientBoostingClassifier(),
    'LinearSVC': LinearSVC()
}

results = {}
for model_name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred, average='weighted', zero_division=0)
    recall = recall_score(y_test, y_pred, average='weighted', zero_division=0)
    f1 = f1_score(y_test, y_pred, average='weighted', zero_division=0)
    results[model_name] = {'Accuracy': accuracy, 'Precision': precision, 'Recall': recall, 'F1-Score': f1}
    print(f"Model: {model_name}")
    print(classification_report(y_test, y_pred, zero_division=0))

results_df = pd.DataFrame(results).T
print(results_df)
```

Fig. 2. Training and Evaluating 7 ML Models

C. Results

The achievement is that Neural Networks and Random Forest exceed the other algorithms in the realization of the accuracy and the F1 score levels. Neural Networks get 55.77% as an absolute maximum due to their learning of difficult patterns outside my typical scheme.

```
Run    Gunay A-A

                    Accuracy  Precision    Recall  F1-Score
Decision Tree       0.557711   0.498776  0.557711  0.466836
Random Forest       0.557790   0.498703  0.557790  0.466367
KNN                 0.517332   0.514539  0.517332  0.492560
Naive Bayes         0.004948   0.522257  0.004948  0.006535
Neural Network      0.556758   0.494571  0.556758  0.467144
Gradient Boosting   0.556626   0.481889  0.556626  0.461320
LinearSVC           0.548079   0.456293  0.548079  0.430202

Process finished with exit code 0
```

Fig.3. The results of the performance metrics for each model in Python

The results of the performance metrics for each model are displayed in the following table:

TABLE II. Results of the Performance Metrics for Each Model

Algorithm	Accuracy (%)	Precision (%)	Recall (%)	F1-Score
Decision Tree	0.557711	0.498776	0.557711	0.466836
Random forest	0.557790	0.498703	0.557790	0.466367

Authorized licensed use limited to: Access provided by VTU consortium. Downloaded on October 02,2026 at 07:51:54 UTC from IEEE Xplore. Restrictions apply.

KNN	0.517332	0.514539	0.517332	0.492560
Naive Bayes	0.004948	0.522257	0.004948	0.006535
Neural Network	0.556758	0.494571	0.556758	0.467144
Gradient Boosting	0.556626	0.481889	0.556626	0.461320
LinearSVC	0.548079	0.456293	0.548079	0.430202

CONCLUSION

ML techniques offer significant advantages over traditional signature-based NIDS, especially in detecting novel attacks. Among the algorithms evaluated, Neural Networks and Random Forests were found to be the most effective for NIDS [8,9]. The selection of the algorithm, however, needs to take into consideration the particular demands and limitations of the deployment scenario.

Future work could explore combining these techniques with deep learning models and real-time detection systems to further improve the robustness and efficiency of NIDS [10].

ACKNOWLEDGMENT

The authors wish to thanks BSc. Aydın Khalilov for edition of the paper in terms of the writing style.

REFERENCES

[1] Gouveia, M. Correia, Network Intrusion Detection with XGBoost, IBM Belgium, 2017.

[2] G. Abdiyeva-Aliyeva, M.Hematyar. "AI-Based Network Security Anomaly Prediction and Detection in Future Network". 4th International Conference on Artificial Intelligence and Applied Mathematics in Engineering. ICAIAME 2022, pages 149-159

[3] Duda, R. O., Hart, P. E., & Stork, D. G. (2000). Pattern Classification. Wiley-Interscience.

[4] L. Podlodowski, M. Kozlowski. Application of XGBoost to the Cybersecurity Problem of Detecting Suspicious Network Traffic Events, 2019 IEEE International Conference on Big Data, 2019. DOI: 10.1109/BigData47090.2019.9006586

[5] Eskin, E. (2000). Evaluation of Anomaly Based Intrusion Detection Systems. Proceedings of the 2000 ACM Symposium on Applied Computing - SAC '00. https://doi.org/10.1145/335603.335635

[6] N. Kaja, A. Shaout, D. Ma. An intelligent intrusion detection system, Applied Intelligence 49 (9) (2019) 3235–3247. doi:10.1007/s10489-019-01436-1.

[7] M. Ahasan, R.Gomes, M.M.Chowdhury, K.E.Nygard. Enhancing Machine Learning Prediction in Cybersecurity Using Dynamic Feature Selector, Journal of Cybersecurity and Privacy, 2021, https://doi.org/10.3390/jcp1010011

[8] G.Abdiyeva-Aliyeva, J.Aliyev, U. Sadigov. Application of classification algorithms of Machine learning in cybersecurity. Procedia Computer Science. Volume 215, 2022, pages 909-919. https://doi.org/10.1016/j.procs.2022.12.093

[9] M. Hajaram N.F. Jumaat, M.A.Bakar. Boosting the Accuracy of Phishing Detection with Less Features Using XGBoost, International Journal of Software & Hardware Research in Engineering, Vol. 8 (2), 2020. https://www.researchgate.net/publication/339727375

[10] C. Bentejac, A. Csorgo, G. Martinez-Munoz. A Comparative Analysis of XGBoost, 2019., pp.3-10.

Authorized licensed use limited to: Access provided by VTU consortium. Downloaded on October 02,2026 at 07:51:54 UTC from IEEE Xplore. Restrictions apply.